

Trigger

Trigger

- Besteht aus Bedingung und Aktion
 - Wenn Bedingung erfüllt, führe Aktion aus
- Alternativen zu Binding-Converter-Kombination

Typen:

- (Property) Trigger
 - Wenn Property bestimmten Wert annimmt
 - Nur DependencyProperty des Ursprungsobjekts, keine .NET Properties
 - Muss in Style definiert werden
- Data Trigger
 - Wie Property Trigger, aber für beliebige Properties beliebiger Objekte
 - Auch für .NET Properties
 - Muss in Style definiert werden
- Event Trigger
 - Sobald ein bestimmtes RoutedEvent auftritt
 - Nur für Storyboards (Animationen) nutzbar
 - Kann direkt in Trigger-Property des Objekts stehen

Trigger ermöglichen das Ändern des visuellen Zustands eines Controls basierend auf bestimmten Ereignissen oder Zuständen. Mit Triggern kann die Darstellung eines Steuerelements dynamisch angepasst werden, indem sie Eigenschaften wie Hintergrundfarbe, Schriftart oder andere visuelle Eigenschaften ändern. Es gibt verschiedene Arten von Triggern in WPF:

1. (Property) Trigger: Property Trigger reagieren auf Änderungen einer bestimmten Eigenschaft eines Steuerelements. Wenn der Wert einer Eigenschaft einen bestimmten Zustand erreicht, wird der Trigger ausgelöst und die im Trigger definierten Änderungen werden angewendet.
2. DataTrigger: DataTrigger reagieren auf Änderungen der Bindung eines Steuerelements. Sie werden normalerweise verwendet, um den visuellen Zustand eines Steuerelements basierend auf den Daten zu ändern, die es anzeigt. Wenn die Daten bestimmte Bedingungen erfüllen, wird der Data Trigger ausgelöst und die entsprechenden Änderungen werden vorgenommen.
3. EventTrigger: EventTrigger reagieren auf das Auftreten eines bestimmten Ereignisses auf einem Steuerelement. Wenn das Ereignis ausgelöst wird, wird der Event Trigger aktiviert und die im Trigger definierten Aktionen werden ausgeführt. Diese sind für gewöhnlich Animationen.

Insbesondere Property- und DataTrigger können unter Umständen als Alternative zu Bindungen verwendet werden, um z.B. Converter-Klassen zu vermeiden. Trigger können allerdings nichts erreichen, was Bindungen nicht auch können.

Property Trigger

- Überwacht eine DependencyProperty des Ursprungsobjekts auf bestimmten Wert
- Wird aktiviert, wenn die Dependency-Property einen bestimmten Wert bekommt
- Muss innerhalb eines Styles/Templates definiert werden
- Setter setzen die neuen Werte, während der Trigger ausgelöst ist

```
<Style TargetType="TextBlock">
  <Style.Triggers>
    <Trigger Property="IsMouseOver" Value="True">
      <Setter Property="Foreground" Value="Red" />
      <Setter Property="TextDecorations" Value="Underline" />
    </Trigger>
  </Style.Triggers>
</Style>
```

Ein (Property) Trigger ist ein Mechanismus, der es ermöglicht, den visuellen Zustand eines Steuerelements basierend auf Änderungen einer bestimmten Controlinternen DependencyProperty zu ändern. Wenn der Wert dieser Eigenschaft einen bestimmten Zustand erreicht, wird der Trigger ausgelöst und es können entsprechende visuelle Änderungen vorgenommen werden.

Um einen Property Trigger zu definieren, wird der Trigger innerhalb des ControlTemplates oder Styles eines Steuerelements erstellt. Er besteht aus zwei Hauptkomponenten: der Eigenschaft, auf die er reagiert, und dem Wert, bei dem der Trigger ausgelöst wird. Wenn der Wert der Ziel-Eigenschaft den Triggerwert erreicht, werden die im Trigger definierten Setter-Aktionen ausgeführt. Dies kann das Ändern von Eigenschaften wie Hintergrundfarbe, Schriftart, Sichtbarkeit oder andere visuelle Eigenschaften umfassen.

Ein Beispiel für einen Property Trigger könnte sein, dass der Hintergrund eines Buttons auf Rot geändert wird, wenn die IsMouseOver-Eigenschaft den Wert "True" hat. In diesem Fall wird der Property Trigger ausgelöst, sobald der Mauszeiger über den Button bewegt wird.

Da mit Property Triggern mit der Änderung mehrerer Eigenschaften auf den Zustand einer Eigenschaft reagiert werden und auf Converter verzichtet werden kann, sind sie eine valide Alternative für simple Binding-Converter-Kombinationen (solange der XML-Parser die entsprechenden Datentypen kennt).

Data Trigger

- Funktioniert wie Property Trigger, kann aber beliebige Eigenschaften überwachen
 - D.h. auch ‚normale‘ .NET-Properties und DependencyPropertyProperties von beliebigen Controls
- Property-Überwachung erfolgt über Binding-Objekt

```
<Label>
  <Label.Style>
    <Style TargetType="Label">
      <Setter Property="Content" Value="Nein"></Setter>
      <Setter Property="Foreground" Value="Red"></Setter>
      <Style.Triggers>
        <DataTrigger Binding="{Binding Path=BoolVal}" Value="True">
          <Setter Property="Content" Value="Ja"></Setter>
          <Setter Property="Foreground" Value="Green"></Setter>
        </DataTrigger>
      </Style.Triggers>
    </Style>
  </Label.Style>
</Label>
<Button x:Name="ChangePropBtn" Click="ChangePropBtn_Click">ChangeNetProp</Button>
```

Nein

ChangeNetProp

Ja

ChangeNetProp

Ein DataTrigger ist ein Mechanismus, der es ermöglicht, den visuellen Zustand eines Steuerelements basierend auf (Daten-)Bindungen zu ändern. Anders als beim Property Trigger, der auf Änderungen von Steuerelementeigenschaften reagiert, reagiert der DataTrigger auf Änderungen der Bindung, welche auch an Eigenschaften des DataContext angebunden werden kann.

Um einen DataTrigger zu definieren, wird der Trigger innerhalb des ControlTemplates oder Styles eines Steuerelements erstellt. Auch der DataTrigger besteht aus zwei Hauptkomponenten: der Binding-Eigenschaft, die auf die Datenquelle verweist, und dem Wert, bei dem der Trigger ausgelöst wird. In allen weiteren Aspekten entspricht die Funktionsweise des DataTriggers der des Property Triggers. Es ist aber zu beachten, dass bei einer Datenbindung auch hier das INotifyPropertyChanged-Interface zu Einsatz kommen muss.

Ein Beispiel für einen DataTrigger könnte sein, dass die Hintergrundfarbe einer ListBoxItem basierend auf einem bestimmten Wert in der zugewiesenen Datenquelle geändert wird. Wenn der Wert in der Datenquelle den Triggerwert erreicht, wird der DataTrigger ausgelöst und die Hintergrundfarbe wird entsprechend geändert.

EventTrigger

- Einziges Trigger-Objekt, das direkt in die „Triggers“-Property der Controls gelegt werden kann
- Wird ausschließlich zur Steuerung von Animationen verwendet

```
<TextBlock Width="200" Height="30" Text="TextBoxHover">
    <TextBlock.Triggers>
        <EventTrigger RoutedEvent="MouseEnter">
            <EventTrigger.Actions>
                <BeginStoryboard>
                    <Storyboard>
                        <DoubleAnimation Duration="0:0:0.300"
                                         Storyboard.TargetProperty="FontSize"
                                         To="28"/>
                    </Storyboard>
                </BeginStoryboard>
            </EventTrigger.Actions>
        </EventTrigger>
    </TextBlock.Triggers>
</TextBlock>
```

Ein EventTrigger ist ein Mechanismus, der es ermöglicht, auf das Auftreten eines bestimmten Ereignisses eines Steuerelements zu reagieren und entsprechende Aktionen auszuführen. Mit einem EventTrigger können Entwickler das Verhalten eines Steuerelements basierend auf Benutzerinteraktionen oder anderen Ereignissen anpassen.

Um einen EventTrigger zu definieren, wird der Trigger innerhalb des ControlTemplates, oder Styles eines Steuerelements oder in dessen eigener Triggers-Eigenschaft erstellt. Im EventTrigger wird das RoutedEvent definiert, auf das reagiert werden soll.

EventTrigger werden ausschließlich zur Steuerung von Animationen innerhalb der sog. Storyboard-Objekte verwendet. Dies erfolgt über Steuerobjekte wie z.B. „BeginStoryboard“. Anders als die anderen Trigger-Typen haben EventTrigger keine automatische Abbruch-Mechanik. Während die anderen Trigger nach Rückänderung der überwachten Eigenschaft das Objekt wieder in den Ausgangszustand versetzen, gibt es bei EventTrigger keine derartige Mechanik.